DOMAIN DRIVEN DESIGN USING JAVA

Documentação do Projeto - 4ª Sprint

Sistema: SOUL UP - Sistema de Gamificação Ambiental

Equipe: TechLeet

Integrantes:

Murilo Munari

Samuel Xavier

Diogo Pedroso

Pedro Henrique

Felipe Amado

Aplicação Spring - API REST

FIAP - 2026

1. Objetivo e escopo

Descrição da solução e funcionalidades identificadas no código-fonte atual.

Objetivo

Estimular hábitos sustentáveis por meio de uma experiência gamificada: o usuário participa de missões ambientais, acumula pontos, compra itens e personaliza seu avatar.

Solução proposta

O SOUL UP disponibiliza uma API REST em Java com Spring Boot, integrada ao Oracle por JDBC. A aplicação organiza as requisições em classes Resource, as regras de negócio em BO, a persistência em DAO e os dados de entrada e saída em TO. O frontend web apresenta a experiência ao usuário.

Principais funcionalidades implementadas

Funcionalidade	Implementação no projeto Spring
Usuários e login	CRUD de usuários; consulta por e-mail no login; cadastro com 100 pontos iniciais; senha omitida no JSON de resposta.
Missões e pontuação	CRUD de missões e vínculos com usuários; controle de período; conclusão via atualização; crédito de pontos e bloqueio de nova conclusão.
Loja e inventário	CRUD de itens; compra pelo inventário com verificação de saldo e posse; desconto de pontos; bloqueio de exclusão de item equipado.
Avatar personalizável	Um avatar por usuário; seis posições de itens; verificação de compatibilidade e de posse dos itens.
Parcerias e códigos	CRUD de parcerias e códigos vinculados a itens e parceiros; validações e datas de parceria.

Limites do escopo atual

Não há endpoint de resgate que entregue um item ao inventário. A aquisição no InventarioBO aceita apenas COMPRA. A tabela Logs está prevista no SQL, sem TO, BO, DAO ou Resource correspondente. O login atual valida e-mail e senha e devolve o usuário; não emite token de sessão.

2. Endpoints REST - parte 1

Base local: http://localhost:8080 | Produção: https://techleet569602.onrender.com

HTTP	URI do recurso	Operação	Status previstos
GET	/techleet/usuario	Listar	200 / 404
GET	/techleet/usuario/{id}	Buscar por ID	200 / 404
POST	/techleet/usuario	Cadastrar	201 / 400
POST	/techleet/usuario/login	Validar login	200 / 400
PUT	/techleet/usuario/{id}	Atualizar	201 / 400
DELETE	/techleet/usuario/{id}	Excluir	204 / 404
GET	/techleet/avatar	Listar	200 / 404
GET	/techleet/avatar/{id}	Buscar por ID	200 / 404
POST	/techleet/avatar	Cadastrar	201 / 400
PUT	/techleet/avatar/{id}	Atualizar	201 / 400
DELETE	/techleet/avatar/{id}	Excluir	204 / 404
GET	/techleet/item	Listar	200 / 404
GET	/techleet/item/{id}	Buscar por ID	200 / 404
POST	/techleet/item	Cadastrar	201 / 400
PUT	/techleet/item/{id}	Atualizar	201 / 400
DELETE	/techleet/item/{id}	Excluir	204 / 404
GET	/techleet/missao	Listar	200 / 404
GET	/techleet/missao/{id}	Buscar por ID	200 / 404
POST	/techleet/missao	Cadastrar	201 / 400
PUT	/techleet/missao/{id}	Atualizar	201 / 400
DELETE	/techleet/missao/{id}	Excluir	204 / 404

Leitura da tabela: os status refletem os retornos explícitos dos Resources. Uma lista vazia normalmente retorna 200 com []. Falhas não tratadas podem resultar em 500. A URI missaoUsuario usa U maiúsculo.

2. Endpoints REST - parte 2

Base local: http://localhost:8080 | Produção: https://techleet569602.onrender.com

HTTP	URI do recurso	Operação	Status previstos
GET	/techleet/inventario	Listar	200 / 404
GET	/techleet/inventario/{id}	Buscar por ID	200 / 404
POST	/techleet/inventario	Cadastrar	201 / 400
PUT	/techleet/inventario/{id}	Atualizar	201 / 400
DELETE	/techleet/inventario/{id}	Excluir	204 / 404 / 400
GET	/techleet/missaoUsuario	Listar	200 / 404
GET	/techleet/missaoUsuario/{id}	Buscar por ID	200 / 404
POST	/techleet/missaoUsuario	Cadastrar	201 / 400
PUT	/techleet/missaoUsuario/{id}	Atualizar	201 / 400
DELETE	/techleet/missaoUsuario/{id}	Excluir	204 / 404 / 400
GET	/techleet/parceria	Listar	200 / 404
GET	/techleet/parceria/{id}	Buscar por ID	200 / 404
POST	/techleet/parceria	Cadastrar	201 / 400
PUT	/techleet/parceria/{id}	Atualizar	201 / 400
DELETE	/techleet/parceria/{id}	Excluir	204 / 404
GET	/techleet/codigo	Listar	200 / 404
GET	/techleet/codigo/{id}	Buscar por ID	200 / 404
POST	/techleet/codigo	Cadastrar	201 / 400
PUT	/techleet/codigo/{id}	Atualizar	201 / 400
DELETE	/techleet/codigo/{id}	Excluir	204 / 404

Leitura da tabela: os status refletem os retornos explícitos dos Resources. Uma lista vazia normalmente retorna 200 com []. Falhas não tratadas podem resultar em 500. A URI missaoUsuario usa U maiúsculo.

3. Diagrama de classes

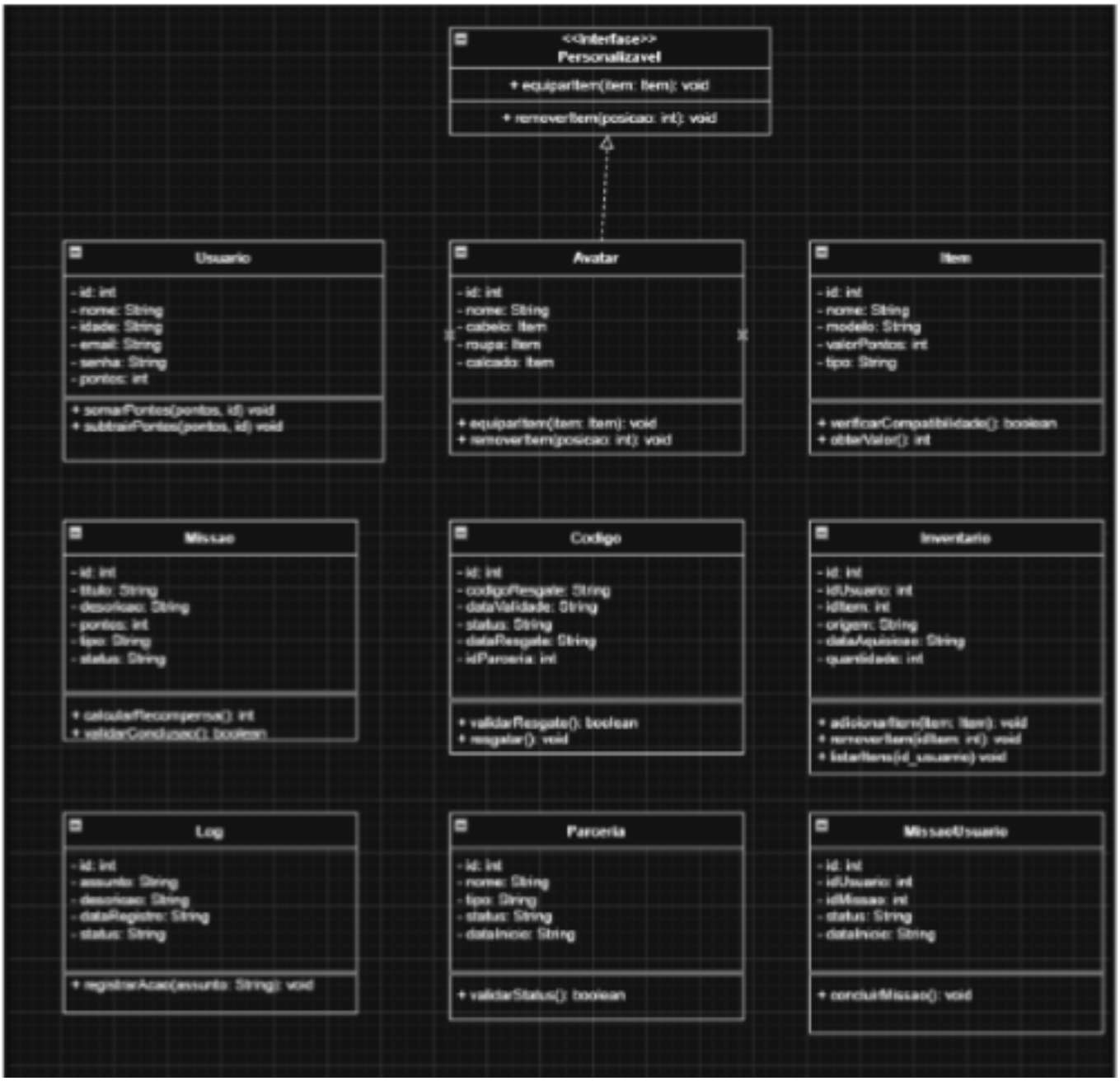
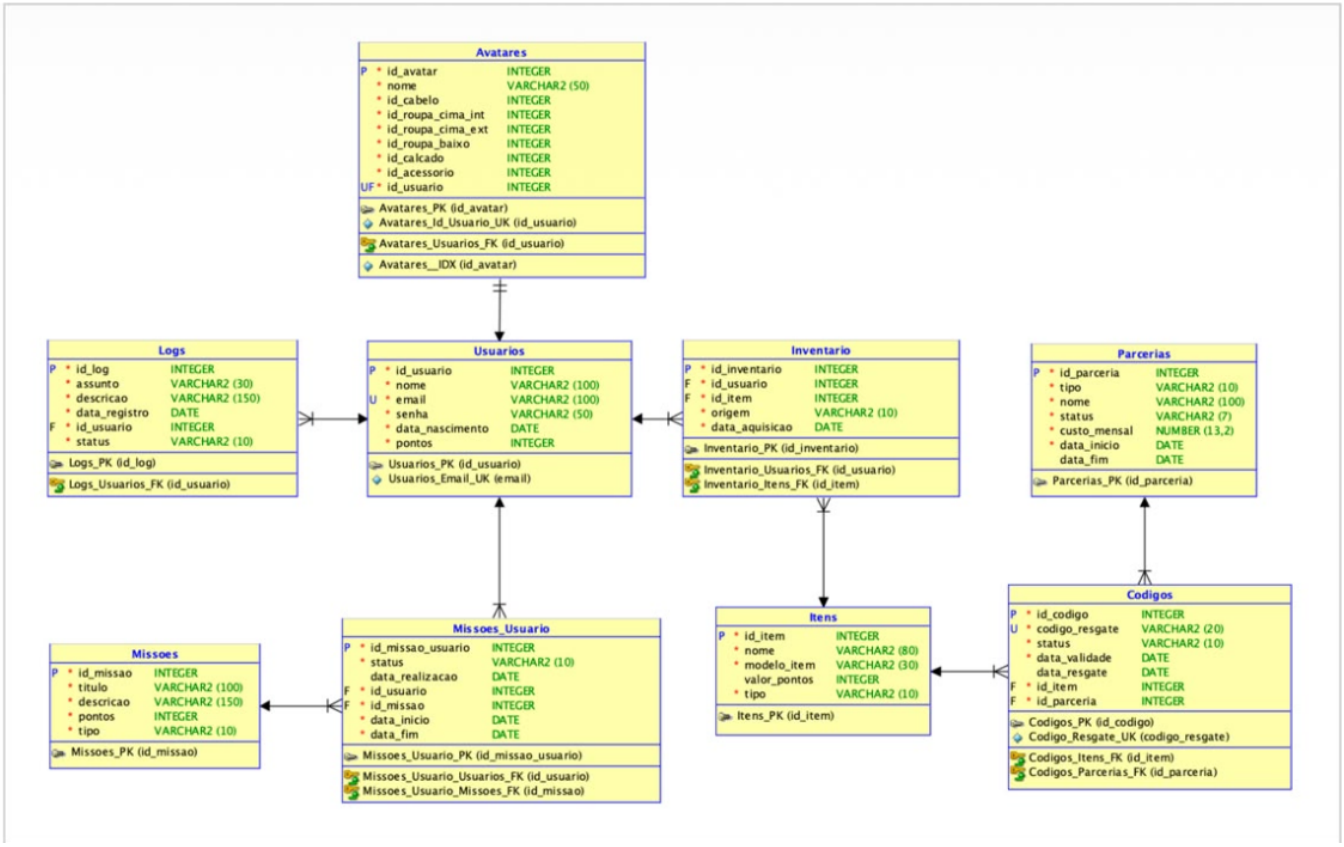


Diagrama de classes fornecido pela equipe, com atributos, métodos e relacionamentos da solução.

4. Modelo Entidade-Relacionamento (MER)



Modelo do banco de dados fornecido pela equipe, com entidades, atributos, chaves e relacionamentos.

5. Ferramentas e configuração

Requisitos extraídos do pom.xml, do Maven Wrapper e da ConnectionFactory.

Ferramenta / requisito	Versão e finalidade
Java JDK	21, configurado em java.version e no Dockerfile. Configure JAVA_HOME para esse JDK.
Spring Boot	4.1.1, conforme o parent do pom.xml. Spring Web MVC e Jakarta Validation compõem a API.
Maven	Wrapper incluso (mvnw / mvnw.cmd); distribuição Maven 3.9.16. A primeira execução precisa acessar o repositório de dependências.
Oracle Database	Persistência dos dados por JDBC. Driver ojdbc17 declarado no pom.xml.
IDE / ferramentas	IntelliJ IDEA para importar o projeto Maven; terminal PowerShell; navegador e Postman ou equivalente para acessar a API.

Configurar a conexão

Defina as variáveis no terminal que executará a aplicação ou em **Run > Edit Configurations > Environment variables** no IntelliJ. Substitua os exemplos pelos dados do ambiente.

```
$env:DB_URL = 'jdbc:oracle:thin:@//HOST:1521/SERVICE_NAME'
$env:DB_USER = 'USUARIO_DO_BANCO'
$env:DB_PASSWORD = 'SENHA_DO_BANCO'
```

A ConnectionFactory lê essas variáveis diretamente. O application.properties atual define apenas o nome da aplicação.

6. Como executar a aplicação

Importação, execução local e verificação inicial.

Pelo IntelliJ IDEA

1. Abra **File > Open** e selecione a pasta Techleet_spring ou o arquivo pom.xml.
2. Importe como projeto Maven e aguarde a resolução das dependências.
3. Em **Project Structure**, selecione o JDK 21. Confira também o JDK usado pelo Maven.
4. Configure DB_URL, DB_USER e DB_PASSWORD na configuração de execução.
5. Execute **br.com.fiap.TechleetApplication** pelo método main.
6. Aguarde a inicialização do servidor e acesse a API em **http://localhost:8080**, se a porta não tiver sido alterada.

Pelo PowerShell

```
git clone https://github.com/TechleetFiap/Techleet_spring.git
cd Techleet_spring
java -version
.\mvnw.cmd -version
# Defina DB_URL, DB_USER e DB_PASSWORD nesta sessão.
.\mvnw.cmd spring-boot:run
```

Gerar e executar o arquivo JAR

```
.\mvnw.cmd clean package
java -jar target/techleet-0.0.1-SNAPSHOT.jar
```

Conferência inicial

Com o servidor iniciado, envie **GET http://localhost:8080/techleet/item**. Com a conexão e as tabelas corretas, a resposta deve ser 200 e um array JSON, que pode estar vazio. No Postman, selecione GET, informe a URL e clique em Send. O caminho raiz / não possui endpoint de apresentação no projeto.

Para testar o frontend publicado, acesse a URL da Vercel indicada ao final. A configuração de apontamento do frontend para uma API local depende do projeto frontend, que não está incluído neste repositório Spring.

7. Repositório e endereços

Código-fonte Spring / GitHub

https://github.com/TechleetFiap/Techleet_spring

API publicada / Render

https://techleet569602.onrender.com

Frontend / Vercel

https://soul-challenge-sprint4.vercel.app/

Exemplos de URIs na API publicada

GET https://techleet569602.onrender.com/techleet/item
GET https://techleet569602.onrender.com/techleet/missao
POST https://techleet569602.onrender.com/techleet/usuario/login